

Ingeniería de características

Lección 6: Creación de características por transformación

Introducción

Seguramente has pensado que además de formar nuevas características por medio de la unión y división de características ya existentes, se pueden generar por medio de la combinación de ellas en otras formas. Piensa un momento en algún conjunto de datos que conozcas que se haya generado en tu trabajo ¿cómo podrías combinar algunas variables para formar otras? Recuerda, siempre con la idea de que estas nuevas características aporten más información al problema que se desea resolver.

Algunas formas son tan simples como aplicar ciertas funciones a las variables existentes, mientras que otras requieren de la transformación completa del conjunto de datos. ¿Cuál es la mejor forma de hacerlo? La respuesta es que no existe una mejor forma, debido a que todo depende de las condiciones del problema que estás tratando de resolver.

Por esta razón, el primer paso es conocer las formas más comunes de estas transformaciones y luego entender completamente el problema en el que se van a aplicar, para así seleccionar la mejor opción con el fin de formar nuevas variables que logren aportar más información al problema.

Escalamiento de características

A continuación, serán examinados los diversos tipos de escalamiento que se pueden utilizar para manejar datos en una misma escala. Por otro lado, también serán revisadas las formas más comunes de transformaciones con el fin de obtener elementos para seleccionar las mejores variables.

Formas

En algunas industrias como la de la construcción, es importante que cuando realizan ciertos procesos de análisis cuenten con bases de datos que contengan la superficie, los metros construidos, el número de cuartos, etc. Para manejar dichos datos de manera fácil deben tener la misma escala y, en la mayoría de los casos esto no es así, requiriendo de un escalamiento de características.

En ocasiones las características **numéricas** con las que se cuentan son las correctas, pero simplemente no están en las unidades adecuadas para utilizarlas de la mejor forma, debido a que las escalas en las que se encuentran presentan una gran disparidad entre ellas. Por ejemplo, si se tiene el siguiente conjunto de datos sobre casas en el frame **casas** de Pandas:

Observa y analiza los datos de la tabla:

```
] casas = pd.read_csv('casas.csv')  
casas
```

```
] 

|   | metros_terreno | metros_construccion | cuartos | estacionamientos |    |
|---|----------------|---------------------|---------|------------------|----|
| 0 | 200            | 130                 | 3       | 2                | 18 |
| 1 | 400            | 312                 | 6       | 3                | 28 |
| 2 | 120            | 120                 | 3       | 1                | 13 |


```

Mientras los metros de terreno y los metros de construcción están dados en cientos, el número de cuartos y el de lugares de estacionamiento están

en unidades. No se diga el precio de la casa, que está en millones. La diferencia de escalas es muy grande.

En algunos procesos que usan los datos, como por ejemplo algunos métodos de *machine learning*, requieren que las escalas de las características sean comunes, es decir, similares, para garantizar un resultado adecuado en un tiempo adecuado. Si las escalas son muy diferentes, algunos de los métodos les cuesta mucho trabajo y tiempo extra el encontrar una solución adecuada.

Para utilizar resultados con escalamientos muy diferentes, la solución es transformarlos de alguna forma en la que todos los datos queden en escalas similares. A este proceso se le conoce como **escalamiento** o **normalización**.

Existen varias formas en las que se puede hacer escalamiento, siendo las más comunes las siguientes:

- **Escalamiento del máximo absolute**

Se obtiene dividiendo cada dato x_i entre el valor máximo de los valores absolutos de los datos en la columna x , lo cual dará un conjunto de valores entre -1 y 1:

$$\frac{x_i}{\max(|x|)}$$

- **Escalamiento min-max**

También llamado **normalización**, se obtiene restando el mínimo valor de la columna x de cada dato x_i y dividiendo el resultado entre la diferencia entre el dato máximo y el dato mínimo de la columna x , llamado **rango** de x . Este escalamiento da como resultado valores entre 0 y 1:

$$\frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- Estandarización

Se logra restando la media de la columna x a cada uno de los datos x_i y dividiendo el resultado entre la desviación estándar σ de la misma columna, lo cual da resultados escalados tanto positivos como negativos:

$$\frac{x_i - \bar{x}}{\sigma}$$

Se cuenta con algunas posibles pequeñas modificaciones de los escalamientos que hacen las operaciones más simples y también dan buenos resultados para algunos problemas, por ejemplo, una modificación que se usa del escalamiento **min-max** es no restar **min**(x) a cada x_i . En general se recomienda utilizar los escalamientos tal como están definidos, debido a que los resultados tienen algunas propiedades estadísticas que muchas veces ayudan a mejorar los resultados al dar escalas mucho más similares.

Aplicación

Para aplicar el escalamiento en un *frame* de Pandas se tienen dos formas comunes:

- Crear una nueva columna cuyos valores se obtienen aplicando la fórmula del escalamiento deseado a los valores de la columna original.
- Utilizar alguna librería como **sci-kit learn** que haga este trabajo automáticamente.

Por ejemplo, si se aplica el escalamiento del **máximo absoluto** a la variable **metros_terreno** de la tabla **casas**, haciendo las operaciones indicadas por la fórmula en forma directa, usando las funciones de Python **max()** para el máximo y **abs()** para el valor absoluto, aplicadas a la columna que queremos escalar. Primero obtenemos el máximo en valor absoluto:

```
maximo = max(abs(casas['metros_terreno']))
```

Luego, al hacer la división de todos los valores de la columna entre el máximo valor encontrado, se deja el resultado en una nueva columna llamada **metros_terreno_norm**, dentro del *mismo frame* **casas**, y se utiliza la instrucción:

```
casas['metros_terreno_norm'] = casas['metros_terreno']/maximo
```

O desde luego, se puede hacer en una sola línea:

```
casas['metros_terreno_norm'] =  
casas['metros_terreno']/ max(abs(casas['metros_terreno']))
```

También se pueden utilizar los métodos **max()** y **abs()** de Pandas para *los frames*:

```
casas['metros_terreno_norm2'] = casas['metros_terreno']/  
casas['metros_terreno'].abs().max()
```

Con los tres casos obtenemos el mismo resultado:

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	metros_terreno_norm
0	200	130	3	2	1825000	0.5
1	400	312	6	3	2850000	1.0
2	120	120	3	1	1375000	0.3

Si se requiere realizar la estandarización, pero ahora usando **min-max**, la instrucción en una sola línea sería la siguiente:

```
casas['metros_terreno_norm'] = (casas['metros_terreno']-
min(casas['metros_terreno']))/(max(casas['metros_terreno'])-
min(casas['metros_terreno']))
```

O bien, con los métodos de Pandas:

```
casas['metros_terreno_norm'] = (casas['metros_terreno']-
casas['metros_terreno'].min())/(casas['metros_terreno'].max()-
casas['metros_terreno'].min())
```

En ambos casos se obtiene la siguiente tabla con la columna normalizada **metros_terreno_norm** colocada al final de la tabla:

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	metros_terreno_norm
0	200	130	3	2	1825000	0.285714
1	400	312	6	3	2850000	1.000000
2	120	120	3	1	1375000	0.000000

Si se decide escalar con estandarización, podemos usar las funciones **mean()** y **std()** que tiene Pandas para obtener la media y la desviación estándar, respectivamente; la instrucción en una sola línea sería:

```
casas['metros_terreno_norm'] = (casas['metros_terreno']-
casas['metros_terreno'].mean())/casas['metros_terreno'].std()
```

Resultando la siguiente tabla, con la columna normalizada **metros_terreno_norm** colocada al final:

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	metros_terreno_norm
0	200	130	3	2	1825000	-0.27735
1	400	312	6	3	2850000	1.10940
2	120	120	3	1	1375000	-0.83205

Estandarización

También se puede realizar escalamiento recurriendo a una librería que ya tenga este tipo de procedimientos implementados en algún método dentro

de ella. Una de las más utilizadas es la librería **sci-kit learn**. Esta librería tiene los tres métodos ya explicados y algunos más, por lo que se recomienda consultar su documentación para conocerlos.

Para obtener un escalamiento con **sci-kit learn** se debe importar el paquete llamado **preprocessing**.

```
from sklearn import preprocessing:
```

Los métodos que contiene para los escalamientos presentados son:

- **MaxAbsScaler**

Para escalamiento máximo absoluto

Para obtener el escalamiento de **máximo absoluto** de la primera columna del *frame* **casas**, llamada **metros_terreno**, se utiliza el método **MaxAbsScaler** para definir el escalador:

```
1: escalador = preprocessing.MaxAbsScaler()
```

y se aplica a todo el *frame*:

```
1: frame_escalado = escalador.fit_transform(casas)
```

Se debe poner mucha atención en que el resultado de esta transformación es una matriz de **numpy** de tres renglones, uno por cada dato, y cuatro columnas, una por cada columna del *frame* **casas**, además de que se aplica el escalamiento a todo el *frame*.

Desde luego que es posible aplicar el escalamiento solo a algunas columnas del *frame*, lo cual se puede indicar en el parámetro, sin embargo, el resultado es siempre una matriz aún y cuando se aplique solo a una columna. La matriz que resulta al aplicarlo a todo el *frame* es:


```
] array([[0.28571429, 0.05208333, 0. , 0.5 , 0.30508475],
        [1. , 1. , 1. , 1. , 1. ],
        [0. , 0. , 0. , 0. , 0. ]])
```

Por lo tanto, para obtener la columna escalada deseada, se selecciona por su posición en la matriz. Debes recordar que la columna 0 es la primera y que se requieren todos los valores de la misma. En este caso, la que se quiere es precisamente la columna 0, que corresponde al escalamiento de la primera columna del *frame* **casas** llamada **metros_terreno**, por lo que la selección de todos sus valores se puede obtener con *slicing*:

frame_escalado[:,0]

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	metros_terr
0	200	130	3	2	1825000	
1	400	312	6	3	2850000	
2	120	120	3	1	1375000	

• MinManScaler

Para escalamiento min-max

Observa que es la misma tabla que se obtuvo con el método donde se realizaron las operaciones indicadas por la fórmula en forma directa.

Si se desea escalar con **min-max** la tercera columna del *frame* **casas** llamada **cuartos**, dejando el resultado en una nueva columna llamada **cuartos_norma**, y suponiendo que ya se importó la librería **preprocessing**, se realizan las siguientes instrucciones:

```
] escalador = preprocessing.MinMaxScaler()
   frame_escalado = escalador.fit_transform(casas)
```



```
]: casas['metros_terreno_norm'] = pd.Series(frame_escalado[:,2])
```

Obteniendo la tabla:

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	cuartos_norm
0	200	130	3	2	1825000	0.5
1	400	312	6	3	2850000	1.0
2	120	120	3	1	1375000	0.5

• StandardScaler

Para escalamiento de estandarización

Si se desea escalar con estandarización la cuarta columna del frame **casa** llamada **estacionamientos**, dejando el resultado en una nueva columna llamada **estacionamientos_norm** y suponiendo que ya se importó la librería **preprocessing**, se realizan las siguientes instrucciones:

```
escalador = preprocessing.StandardScaler()
frame_escalado = escalador.fit_transform(casas)
```

```
]: casas['metros_terreno_norm'] = pd.Series(frame_escalado[:,3])
```

Obteniendo la tabla:

	metros_terreno	metros_construccion	cuartos	estacionamientos	precio	cuartos_norm
0	200	130	3	2	1825000	0.000000
1	400	312	6	3	2850000	1.224745
2	120	120	3	1	1375000	-1.224745

Como un último comentario se debe decir que en varios de los métodos de *Machine Learning* donde se usa escalamiento, la variable de salida se puede dejar sin escalar, ya que sería la variable dependiente de la función.

Uso de funciones

Una forma de transformación de características es la aplicación de ciertas funciones que logran acentuar las que pertenecen a ciertas variables, siendo mejor aprovechadas. Con base en lo anterior, a continuación serán examinadas diversas funciones, tales como Potencias, Logaritmos y *One-Hot Encoding*.

Otra forma de transformación de características muy común con la que contamos es la aplicación de ciertas funciones que logran acentuar algunas características de las variables que pueden ser mejor aprovechadas por el método que está utilizando los datos. Ten en cuenta que las columnas son las variables independientes de las funciones y el resultado es el que formará la nueva columna del frame.

El número de funciones que podemos aplicarle es infinito. Básicamente, cualquier función que se te ocurra puede ser aplicada a las columnas, es más, algunas funciones de dos o más variables pueden ser aplicadas a dos o más características para formar una nueva. Sin embargo, para seleccionar la función adecuada debes recordar que el objetivo de este tipo de transformación es que se obtengan nuevas variables cuya información o características puedan ser mejor aprovechadas por los métodos que las están usando.

Hay funciones que han dado buenos resultados en varios problemas específicos y que servirán como base para mostrarte cómo se hacen estas transformaciones y aprendas cómo aplicarlas para crear nuevas variables.

A continuación, se procederá a ver algunas de ellas.

Potencias

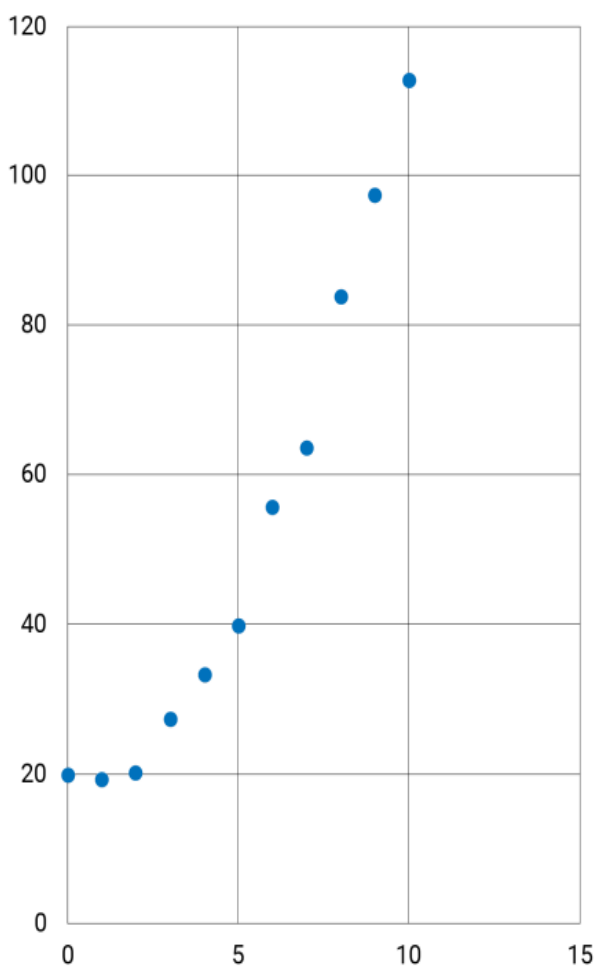
En algunos problemas la relación entre una variable de entrada x y una de salida y , no se puede aproximar usando funciones lineales de la forma:

$$y = x$$

A veces la relación entre ellas no es lineal sino de un grado k mayor. En estos casos, para aproximar mejor la relación entre la variable de entrada y la de salida, será necesario usar una función no lineal. Una de las formas que se utilizan es la de un **polinomio de grado k** de la forma:

$$y = w_0 + w_1x + w_2x^2 + \dots + w_kx^k$$

Donde las w_i son constantes reales. Si el conjunto de datos proporcionado solo contiene una variable x y una salida y , será necesario formar las potencias restantes de dicha variable, cada una de las cuales será una nueva variable. Por ejemplo, si se tiene el *frame* **puntos**, que contiene el siguiente conjunto de puntos como datos:



Si se grafica, es posible observar que la relación entre x y y no es lineal, parece más una parábola, es decir, una relación cuadrática:

Con esto se sabe que la relación podría ser de la siguiente forma:

$$y = w_0 + w_1x + w_2x^2$$

Donde, desde luego, faltaría conocer los valores exactos de las constantes. Para encontrar estos valores que logren aproximarse a los puntos lo mejor posible, se puede hacer una regresión, pero, para que la función pueda ser un polinomio de grado 2, es necesario crear una nueva variable a partir de la x con la que se cuenta. La nueva columna llamada x^{**2} se formaría en Pandas con la instrucción:

`puntos['x**2'] = puntos['x']**2`

Obteniendo la nueva tabla:

	x	y	x**2
0	0	19.8843	0
1	1	19.2282	1
2	2	20.0582	4
3	3	27.3267	9
4	4	33.1432	16
5	5	39.7118	25
6	6	55.5618	36
7	7	63.5668	49
8	8	83.7457	64
9	9	97.2570	81
10	10	112.5323	100

Y ahora sí se podría correr la regresión con una función cuadrática.

Si la relación fuera cúbica, se requeriría la creación de una variable más X^3 de la siguiente forma:

$$\text{puntos}[x^{**3}] = \text{puntos}[x]^{**3}$$

Y así las variables que hicieran falta de acuerdo al grado del polinomio.

Si se tuvieran dos variables de entrada, X_1 y X_2 , y una de salida y, para generar una función cuadrática se tendría que usar un polinomio cuadrático de dos variables:

$$y = w_0 + w_1X + w_2X^2 + w_3X_1^2 + w_3X_2^2 + w_4X_1X_2$$

Y se tendrían que formar 3 variables: X_1^2 , X_2^2 y X_1X_2 , de la siguiente forma:

$$\text{puntos}[x_1^{**2}] = \text{puntos}[x_1]^{**2}$$

$$\text{puntos}[x_2^{**2}] = \text{puntos}[x_2]^{**2}$$

$$\text{puntos}[x_1x_2^{**2}] = \text{puntos}[x_1] * \text{puntos}[x_2]$$

Es importante que conozcas que todas las variables de un polinomio se forman fácilmente con potencias y multiplicaciones de las variables existentes.

Logaritmos

Algunos requieren acentuar las diferencias entre dos valores muy parecidos, donde dichas diferencias son muy pequeñas. Los logaritmos son expertos en hacer esa función.

A continuación, se muestra un ejemplo:

Si se quiere obtener un número cercano a 1 y se obtiene un 0.98, la diferencia es poca, en cambio, si se obtiene un 0.49, la diferencia es mucho mayor; si se obtiene un 0.1, la diferencia es muy grande. Sin embargo, si se resta de 1 estas cantidades se obtiene:

$$1 - 0.98 = 0.02$$

$$1 - 0.49 = 0.51$$

$$1 - 0.1 = 0.9$$

En realidad, todos los resultados son muy pequeños porque son menores a 1. Si se necesita remarcarlos más, se les puede aplicar el negativo del logaritmo base 10 en lugar de la resta:

$$-\log(0.98) = 0.0088$$

$$-\log(0.49) = 0.3098$$

$$-\log(0.1) = 1.0000$$

$$-\log(0.01) = 2.0000$$

Puedes observar que entre más alejando esté de 1, más se hace grande el resultado y entre más cerca, más pequeño. En otras palabras, la función logaritmo ayuda a remarcar esta diferencia.

Hay logaritmos de diferentes bases. El logaritmo base ***b*** de un número ***x*** es el exponente ***k*** al que se debe elevar la base ***b*** para obtener el número ***x***. Es decir:

$$\text{Si } \log_b(x) = k, \text{ entonces } b^k = x$$

La librería ***numpy*** de Phyton tiene implementados los procedimientos para obtener tres logaritmos muy usados:

Log2(): logaritmo base 2

Log10(): logaritmo base 10

Log(): logaritmo natural o base $e = 2.71828\dots$, que es un número irracional

Su aplicación es muy simple. Si se quiere obtener una nueva característica llamada **log10X** para el frame **puntos** que sea el logaritmo base 10 de una ya existente, como **x**, primero se debe importar la librería **numpy**:

Import numpy as np

Y luego aplicar el logaritmo base 10 **log10()** que ya tiene implementado:

Puntos['log10X'] = np.log10(puntos['x'])

Da como resultado el nuevo *frame* puntos:

	x	y	log10X
0	0	19.8843	-inf
1	1	19.2282	0.000000
2	2	20.0582	0.301030
3	3	27.3267	0.477121
4	4	33.1432	0.602060
5	5	39.7118	0.698970
6	6	55.5618	0.778151
7	7	63.5668	0.845098
8	8	83.7457	0.903090
9	9	97.2570	0.954243
10	10	112.5323	1.000000

Debes tener cuidado al aplicar esta función porque **no existe para 0** ni para números negativos. Puedes observar que en el renglón 0 da como resultado **-inf**, el cual se puede limpiar antes o después, pero lo importante es que lo detectes.

Generalmente, estos tres son suficientes para todas las aplicaciones, pero si te hace falta un logaritmo de otra base **c**, este se puede obtener fácilmente a partir de un logaritmo base **b** ya conocido, con la relación:

$$\log_c(x) = \frac{\log_b(x)}{\log_b(b)}$$

One-Hot Encoding

La codificación *One-Hot* es una forma especial que ayuda a representar números enteros utilizando un vector binario. En este vector, todos sus elementos son 0, excepto uno que tiene un 1 y que representa la posición

del número entero que se está codificando. Los números a codificar deben ser enteros consecutivos e inician a partir del 0, aunque se puede hacer un mapeo local para cambiarles el nombre. Sin embargo, para efectos de esta explicación se puede suponer que siempre son números enteros consecutivos iniciando en 0. Si se cumplen estas condiciones, se van a requerir tantos bits en el vector como números se quieran representar.

¿Un poco confuso? No hay problema, con un ejemplo quedará completamente claro ya que es muy simple.

- **Ejemplo**

Si se desean codificar los números {0, 1, 2, 3} en *One-Hot* cada número va a estar representado por un vector de 4 bits, donde el bit de más a la derecha es el primero y solo hay un 1, todos los demás son 0. La codificación para el 0 es 0001, lo que significa que el entero es el primero del conjunto, porque el bit que es 1 es el primero, esto es, el de más a la derecha. La codificación para el 1 es 0010, la del 2 es 0100 y la del 3 es 1000.

Si se realiza alguna relación con los enteros, la representación One-Hot también puede ser utilizada para **variables categóricas**. Por ejemplo, si dada una foto se tiene que decidir si es **manzana**, **pera** o **fresa**, se puede usar un 0 para **manzana**, un 1 para **pera** y un 2 para **fresa**, o con alguna otra asignación. De esta forma, en One-Hot el vector 001 representaría una **manzana**, 010 una **pera** y 100 una **fresa**.

La codificación One-Hot es muy utilizada normalmente para la salida y solo en algunos procesos se usa este tipo de datos como, por ejemplo, las Redes Neuronales cuando son usadas para hacer clasificación. Resulta que las Redes Neuronales interpretan más fácilmente las salidas One-Hot que directamente los enteros.

Aunque el concepto es sencillo, la transformación de un entero a su representación One-Hot requiere de cierto procesamiento que solicita conocer un poco de programación. Este proceso recibe como entrada un entero y da como salida su representación One-Hot, es decir, un vector

binario. Este vector binario puede ser escrito de muchas formas: un string como '0100', una lista de enteros como [0,1,0,0], un arreglo de **numpy**, o usar valores booleanos False y True, en lugar de 0 y 1, entre otras. La forma de la salida depende de lo que requiera el problema.

Afortunadamente, algunas librerías para hacer análisis de datos, como **scikit-learn** o Pandas tienen implementado un método que hace precisamente esto.

Si se tiene un frame llamado **fruta** con los siguientes datos:

	precio	color	id	clase
0	10.25	rojo	0	manzana
1	12.50	verde	1	pera
2	21.50	rojo	2	fresa
3	13.42	verde	0	manzana
4	10.20	amarillo	1	pera
5	15.40	amarillo	0	manzana

Observa que se tiene una variable clase que es categórica en el conjunto {manzana, pera, fresa}. También se tiene el id de la fruta, que corresponde a un valor entero {0,1,2} respectivamente.

Pandas usa la función **get_dummies()**, que toma una **variable categórica**, y regresa tantas columnas como categorías tenga la columna. Los nombres brindados a las columnas son los de las categorías, colocándose en orden alfabético. Se asigna cero en todas ellas y solo un 1 en la que le corresponda. La instrucción:

fruta_OH = pd.get_dummies(fruta.clase, prefix = 'clase')

Dando como resultado la siguiente tabla:

	clase_fresa	clase_manzana	clase_pera
0	0	1	0
1	0	0	1
2	1	0	0
3	0	1	0
4	0	0	1
5	0	1	0

El prefijo indicado en ***prefix*** es opcional. Se acostumbra a poner el nombre de la columna original; este prefijo se pega a los nombres de las categorías con un guion bajo.

La interpretación es tal como se expuso anteriormente. Por ejemplo, el renglón 1, como tiene solo un 1 debajo de la columna **pera**, significa que 001 representa una **pera**. Estas nuevas columnas se pueden pegar al *frame* original borrando la columna clase.

Estas nuevas columnas se pueden pegar al *frame* original borrando la columna clase.

Otra opción para obtener la codificación *One-Hot* es usar la librería **scikit-learn**. Dentro del paquete **preprocessing** se tienen las funciones **OneHotEncoder()** y **LabelBinarizer()** para realizar esta codificación.

Primero, se debe importar la función del paquete:

```
from sklearn.preprocessing import LabelBinarizer
```

y simplemente se aplica a la variable deseada:

```
fruta_OH = LabelBinarizer().fit_transform(fruta.clase)
```

dando como resultado un arreglo de **numpy**:

```
array([[0, 1, 0], [0, 0, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1], [0, 1, 0]])
```

Observa que los renglones son los mismos que los que se obtuvieron con pandas. Para hacerlo con la otra función, primero se importa del paquete:

```
from sklearn.preprocessing import OneHotEncoder
```

Y luego se aplica a la columna deseada, con la única diferencia de que debe recibir una matriz, no un vector, por lo que si es una columna se debe convertir en un arreglo de dos dimensiones, lo cual se logra fácilmente con

dos corchetes y regresa un elemento de una clase especial que se puede convertir a matriz con el método **toarray()**:

```
fruta_OH = OneHotEncoder().fit_transform(fruta[['clase']]).toarray()
```

Que también regresa un arreglo de **numpy**, pero de *floats*:

```
array([[0., 1., 0.], [0., 0., 1.], [1., 0., 0.], [0., 1., 0.], [0., 0., 1.], [0., 1., 0.]])
```

Estos arreglos se pueden colocar en columnas en el *frame* original, borrando la columna que se convirtió.

Reducción de dimensiones

A continuación se analizará el tema de la Reducción de dimensiones, especialmente a través del método de Análisis de Componentes Principales. Se debe destacar que el método mencionado consiste en una proyección de los datos en un espacio con menos cantidad de dimensiones.

¡Vamos a revisarlo!

Introducción

Piensa en una tabla de datos. Recuerda que contiene una serie de variables también conocidas como características, en sus columnas. Cada una de las n variables de ese conjunto de datos se pueden interpretar también como una dimensión en un espacio n -dimensional, y cada uno de los renglones como puntos en ese espacio.

¿En qué consiste la reducción de dimensiones?

También llamado ***Dimensionality Reduction***, consiste en transformar todo el conjunto de datos de un espacio de alta-dimensión a un espacio de baja-dimensión, reduciendo el número de variables que lo representan, pero manteniendo las propiedades más importantes que describen el conjunto de datos original.

Análisis de componentes principales

Existen varios métodos para hacer la reducción de dimensiones y uno de los más utilizados es la **selección de características** o *feature selection*. Sin embargo, el método más común para hacer la reducción de dimensiones en forma automática es el llamado **Análisis de Componentes Principales** o **PCA** por su nombre en inglés, *Principal Components Analysis*, el cual hace una proyección de los datos en un espacio con menos dimensiones.

¿Sabías qué...?

La teoría detrás de PCA tiene que ver con métodos del **Álgebra Lineal** para descomponer una matriz en las partes que la constituyen, sin embargo, se puede pensar en estos métodos como transformaciones del conjunto original de datos de **n** variables a **n** o menos variables completamente nuevas. Estas **n** nuevas variables son evaluadas con alguna métrica y se presentan de la mejor a la de peor calificación. Generalmente, la métrica usada es qué tan bien **explica la varianza** del conjunto original de datos, por lo que los que tengan valores cercanos a 0 pueden ser eliminados. Lo más importante de este nuevo conjunto de datos es que mantiene las características principales del conjunto original, aunque no se puedan detectar a simple vista. Finalmente, se selecciona un subconjunto de tamaño **m** de estas nuevas variables para ser utilizado en un proceso específico que iba a utilizar los datos originales.

Como siempre, es mejor verlo trabajar con un ejemplo e irlo explicando de tal forma que las cuestiones muy técnicas queden a un lado para darle mayor importancia al proceso. Afortunadamente, **scikit-learn** tiene implementado este método en **PCA()**.

• Ejemplo

Si suponemos que se tiene un *frame* llamado **puntos2D** que contiene los siguientes datos:

	x1	x2
0	1	3
1	8	17
2	9	19
3	6	13
4	2	5

En realidad, la variable **x2** es una función lineal de la variable **x1**:

$$\mathbf{x2} = 2 * \mathbf{x1} + 1$$

Por lo anterior, no se está aportando ninguna información adicional al conjunto de datos. Esto lo debes recordar para entender los resultados del

PCA que se hará. Para obtener sus PCA de este conjunto de datos, primero se importa **PCA()** del paquete **decomposition**:

```
from sklearn.decomposition import PCA
```

Se crea una instancia de PCA y se indica el número de nuevas variables que se requieren, en este caso se van a dejar 2, como en el conjunto original, si no le pone nada, también toma todos, pero podrían ser menos:

```
pca = PCA(2)
```

Se aplican a los datos del *frame*:

```
pca.fit(puntos2D)
```

Ahora la variable **pca** contiene los componentes principales y su evaluación, es decir, qué tanta varianza explica cada uno. Estos valores se pueden apreciar al imprimir dos variables de instancia que tiene **pca**, **components_** y **explained_variance_**. Primero se imprimen los componentes que encontró:

```
print(pca.components_)
```

Que da como resultado la impresión de los componentes en una matriz de 2 x 2 porque se le pidieron 2 componentes:

```
[[-0.4472136 -0.89442719]  
 [ 0.89442719 -0.4472136 ]]
```

Y la evaluación de los dos componentes obtenidos:

```
print(pca.explained_variance_)
```

Que da como resultado:

```
[6.35000000e+01 1.36566344e-32]
```

Eso significa que, de la suma de esa lista, el primer componente explica 63.5 puntos de la varianza y el segundo 1.36×10^{-32} , que es casi cero. Es decir, casi toda la información de este conjunto de datos puede ser obtenida con el primer componente y el segundo no contiene mucha información. A veces, es más comprensible conocer el porcentaje de varianza que está explicando cada componente. Este porcentaje se obtiene sumando todos los valores y dividiendo cada uno entre el total, pero esto se puede lo podemos obtener al imprimir la variable **explained_variance_ratio_**:

```
print(pca.explained_variance_ratio_)
```

Lo que nos da como resultado:

```
[1.00000000e+00 2.15065108e-34]
```

Que significa que el primer componente explica casi el 100%, mientras que el segundo explica el $2.15 \times 10^{-32}\%$, esto es, prácticamente 0%.

Finalmente, para formar las nuevas variables se deben aplicar estos componentes al conjunto original de datos y esto se hace con el método **transform()**:

```
puntos2DPCA = pca.transform(puntos2D)
```

Dando como resultado el siguiente arreglo:

```
array([[ 9.39148551e+00,  4.44089210e-16],  
       [-6.26099034e+00, -4.44089210e-16],  
       [-8.49705831e+00, -4.44089210e-16],  
       [-1.78885438e+00, -1.11022302e-16],  
       [ 7.15541753e+00,  4.44089210e-16]])
```

Este arreglo contiene las nuevas *features*, es decir, las nuevas variables, donde la primera, colocada en la primera columna, explica casi al 100% los datos originales y solo se puede quedar con ella como la nueva *feature*, en lugar de las dos originales. La segunda columna está compuesta prácticamente por puros ceros.

El problema de este método es que no es posible explicar en qué consisten las nuevas variables, es decir, es fácil ver que en el *frame* original la variable x_1 y la x_2 representan un punto, por ejemplo, en un plano cartesiano, sin embargo, las nuevas variables no tienen una interpretación directa. Simplemente, se debe confiar en que las nuevas variables seleccionadas pueden sustituir sin ningún problema al conjunto original de variables; generalmente, les damos el nombre de **pca1**, **pca2**, etc.

Desde luego, este método de los componentes principales se puede aplicar solo a una parte del *frame* de datos. Al terminar el proceso, esta parte del *frame* original puede ser sustituido con las nuevas variables obtenidas, aunque no tengan una interpretación directa de lo que representan.

El número de las nuevas variables que se deben seleccionar son aquellas que expliquen un porcentaje adecuado de la varianza. Este porcentaje depende totalmente del problema que se quiere resolver y tú como científico de datos tienes la responsabilidad de decidirlo. Un valor común es 90% a 99%, pero hay casos en los que se puede aceptar un porcentaje menor, dependiendo del problema.

Ideas para llevar

La creación de nuevas características mediante la transformación de las existentes es una práctica muy común que tengo que realizar como científico de datos.

Para crear una nueva característica debo estar seguro de que aportará más información a la salida, lo que ayudará a resolver el problema de una mejor forma.

En este tema logré:

- Identificar que existen varias formas de transformar las características existentes y así generar nuevas. Debo usar la que más convenga a una cierta situación específica.
- Reconocer que, hasta ahora no existe una forma que garantice que las características creadas siempre van a ser mejores que las existentes. No queda otra más que probar varias opciones y realizar la evaluación o la prueba en el proceso en el que se utilizarán los datos.
- Identificar que existe un gran número de métodos para hacer transformaciones de características con el propósito de crear mejores. Muchos de ellos ya están implementados en los paquetes de ciencias de datos y otros se están creando. Debes estar enterado de los existentes y atento a las novedades.
- Reconocer que la experiencia y la práctica me lleve a pensar en una nueva transformación que de mejores resultados para un problema particular que las ya existentes. Cuando eso suceda no debo olvidar documentarlo y seguirlo probando para tenerlo listo para su publicación en foros especializados. Esto ayuda al desarrollo de la Ciencia de Datos, existiendo siempre la posibilidad de crear nuevas librerías donde se implementen estos nuevos métodos para que la comunidad los pueda usar.

- Reconocer que como científico de datos es una buena idea manejar muy bien un lenguaje de programación y conocerlo por completo. Esto implica estar atento a la aparición de nuevas librerías y versiones de las mismas.

Material adicional

Bibliografía

Los contenidos de esta lección están basados en la siguiente bibliografía:

- Kuhn, M. & Johnson, K. (2020). *Feature Engineering and Selection*. CRC Press
- Zheng A. & Casari, A. (2018). *Feature Engineering for Machine Learning Principles and Techniques for Data Scientists*. Oreilly